Incremental Extraction From MySQL Database

Explore incremental extraction methods to efficiently extract only newly added or updated records from MySQL databases. Understand how to use timestamps or status flags to filter data, minimizing load and ensuring your ETL pipeline transfers only relevant recent transactions to data warehouses.

Full extraction of an entire table or database can be very rare. In most cases, users extract only what they need from the databases, which is usually the most recent data.

In **incremental extraction**, only records from the source table that have changed or been added will be extracted. The implementation details of this will vary from use case to use case, but the general idea is to use metrics such as `last_update_date` or `is_active` to filter records that have been recently changed and extract only those.

Incremental extraction is a very common method for transferring data from a transactional OLTP database into a data warehouse. We can use incremental extraction to get only the recent transactions from the production database and transfer only those to the data warehouse.

Example

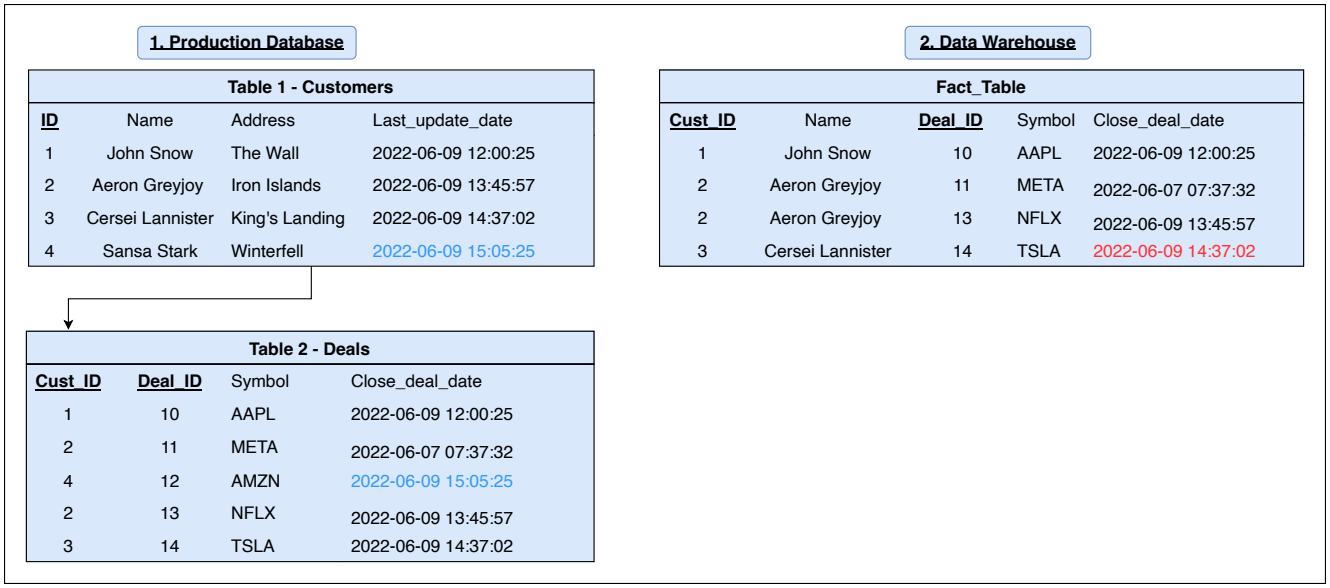
Suppose we're in charge of transferring recent activities and transactions f production database of a stock investment company into a data warehouse for analysis and auditing reports.

Ask

To simplify this example, let's say that the production database is made of two tables `Customers` and `Deals`. The data warehouse is made of a single table named `Fact_Table`.

Our task is to create an ETL pipeline that extracts the most recent data from the `Production` database.

To know exactly which records we should extract, we use the column `Close_deal_date` from the data warehouse to check the last time the ETL pipeline loaded data into the data warehouse.



Investment company components: Production database, data warehouse

We then rely on that information during the next run of the pipeline to select the correct records from the production database.

Ask

Investment company components: ETL pipeline

This is an example of incrementally extracting data from the production database instead of doing a full extraction.

Extracting the last `Close_deal_date` from the data warehouse

main.sql MySQL

```sql
-- Creating the fact table
create table fact_table(
  Cust_ID int not null,
  Name varchar(30) not null,
  Deal_ID int not null,
  Symbol varchar(10) not null,
  Close_deal_date datetime not null,
  primary key(Cust_ID, Deal_ID)
);

-- Inserting historical transactions
insert into fact_table(Cust_ID, Name, Deal_ID, Symbol, Close_deal_date)
values
```

Run

Extracting the last extraction date from the data Warehouse

So the most recent record in the data warehouse is a deal that was closed on `2022-06-09` at `14:37:02`. That is our last extraction date. Let's use that to select only the relevant records from the production database.

Extracting the relevant records from the production database

We'll extract deals that were closed *after* the last extraction date by running the following code snippet:

Ask

main.sql MySQL

```sql
-- Extracting deals that were closed AFTER the last_extraction_date
select Cust_ID, Name, Deal_ID, Symbol, Close_deal_date
from customers as c
join deals as d on
  c.ID = d.Cust_ID
where
  d.Close_deal_date > '2022-06-09 14:37:02';
```

Run

Extracting the latest deals from the production database

Looks like there's only one deal that was closed after `2022-06-09 14:37:02`. Therefore, that is the record that we'll transfer to the data warehouse.

Note that this is just one example of incremental extraction. As we might expect, this heavily depends on the business context. A marketing company, for example, might choose to extract relevant users by other metrics such as `amount_of_clicks`, `is_active`, `number_of_logins`, etc.

Mark As Completed

Ask